

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Arquitectura Cliente-Servidor

Grupo: 02 - Semestre: 2027-1

Tarea 5:

Sniffer de red en Linux.

Profesor:

MTRO. CARLOS ALBERTO ROMAN ZAMITIZ

Fecha de entrega:

04 de Noviembre del 2026

1. Preparación del sniffer

Primero se revisaron las interfaces de red y se activó el modo promiscuo en `lo`, ya que el cliente y el servidor se ejecutaron en el mismo equipo. En la terminal con permisos de root se utilizaron los siguientes comandos:

```
ip -br link
ip link set lo promisc on
ifconfig lo
gcc sniffer.c -o sniffer
./sniffer
```

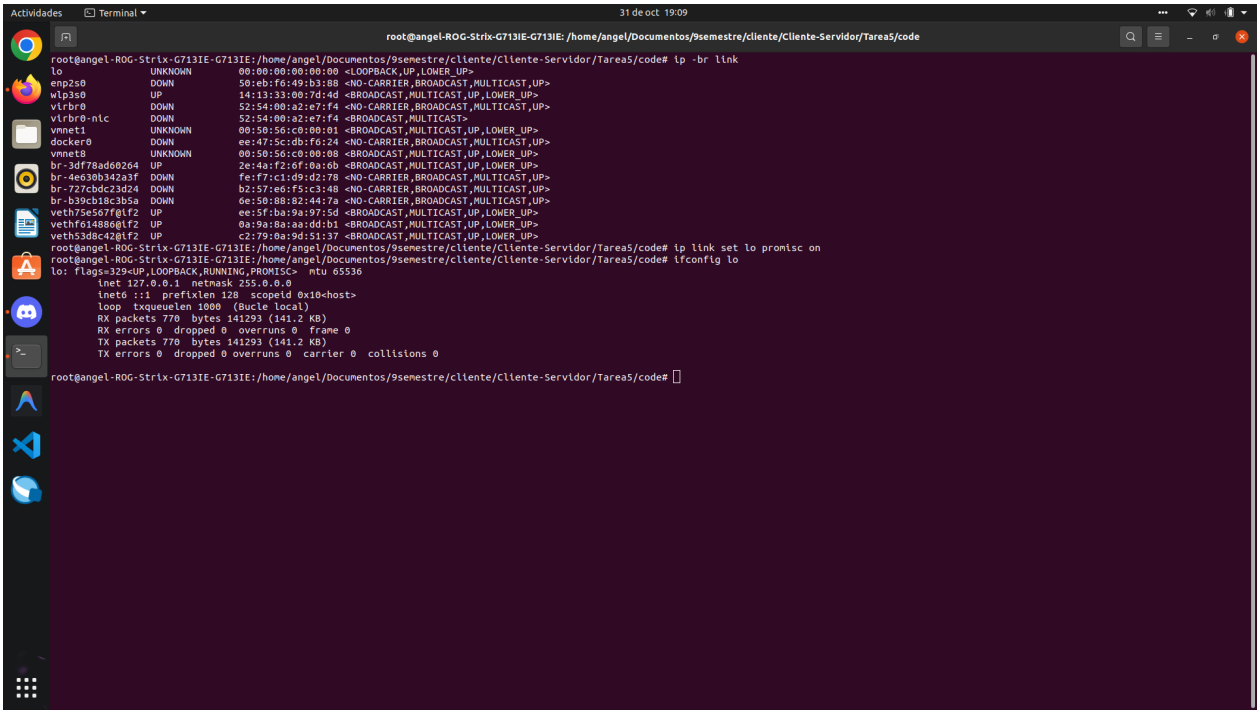


Figura 1: Preparación de la interfaz lo y comprobación del modo promiscuo en Linux.

Lo que vemos aquí es que `ifconfig lo` muestra la bandera `PROMISC` y la dirección `127.0.0.1`. Después se levantó el sniffer en esa misma terminal para esperar el tráfico del puerto 4898.

2. Prueba con el programa 04

Para esta primera prueba se utilizó el servidor que envía automáticamente una frase al cliente. Se compiló cada programa por separado y los ejecutables se nombraron `servidor` y `cliente`. Con el sniffer activo, se levantó primero el servidor y después se ejecutó el cliente en otra terminal:

```
./servidor 4898
./cliente 127.0.0.1 4898
```

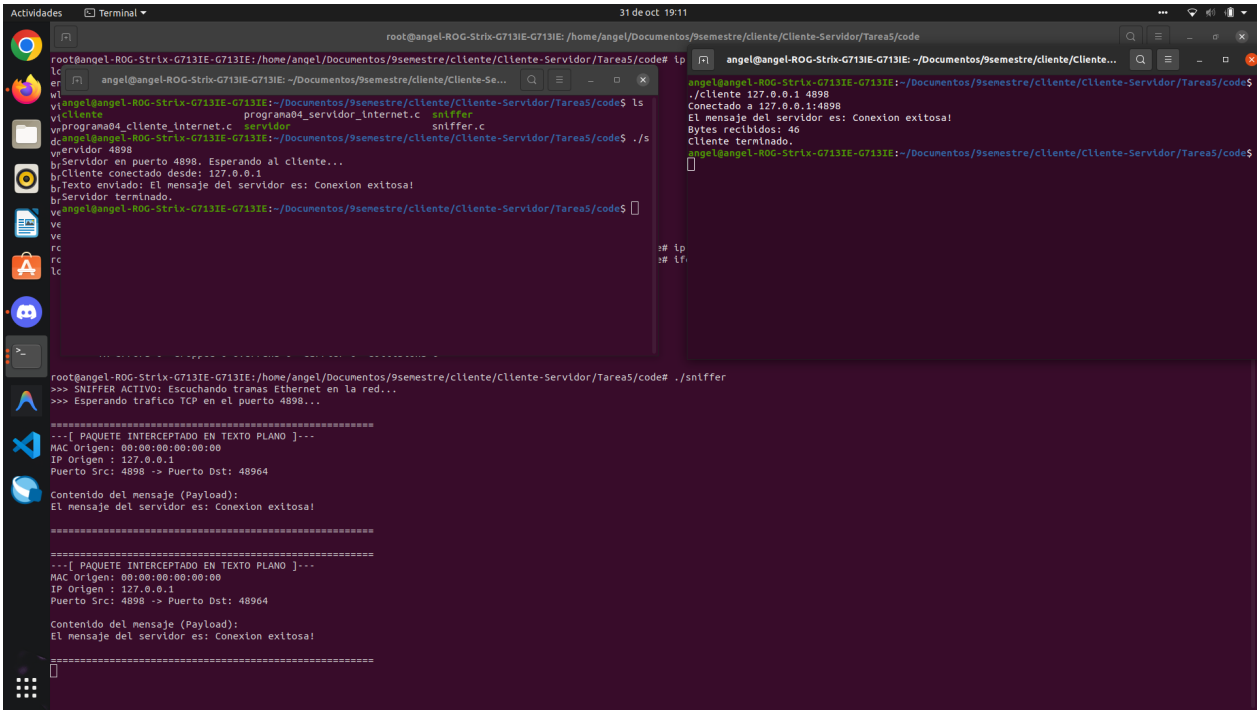


Figura 2: Prueba del programa 04 con las terminales del servidor, cliente y sniffer.

Lo que vemos en la terminal de la izquierda es que el servidor aceptó al cliente desde `127.0.0.1` y envió la frase `El mensaje del servidor es: Conexion exitosa!`. En la terminal de la derecha, el cliente mostró el mismo mensaje y registró 46 bytes recibidos. Después ambos programas terminaron.

Por otra parte, en la terminal del sniffer aparece esa misma frase dentro del payload. El puerto de origen fue 4898 y el de destino 48964, que corresponde al puerto del cliente en esta conexión. Con esto se comprobó que el mensaje enviado por el servidor se podía leer en texto plano.

Además de eso, la dirección MAC aparece con ceros y el paquete se muestra dos veces. Esto corresponde a la captura en loopback; no significa que el servidor haya ejecutado dos veces el envío de la frase.

3. Prueba con el programa 05

Ahora bien, para la segunda prueba se utilizó el par de programas 05, donde el cliente y el servidor pueden intercambiar varios mensajes. Se compilaron estos archivos para generar nuevamente `servidor` y `cliente`, y se mantuvo el mismo puerto:

```
gcc programa05_servidor_internet.c -o servidor
gcc programa05_cliente_internet.c -o cliente
./servidor 4898
./cliente 127.0.0.1 4898
```

Los comandos del servidor y del cliente se ejecutaron en terminales distintas. El sniffer permaneció en una tercera terminal con permisos de root.

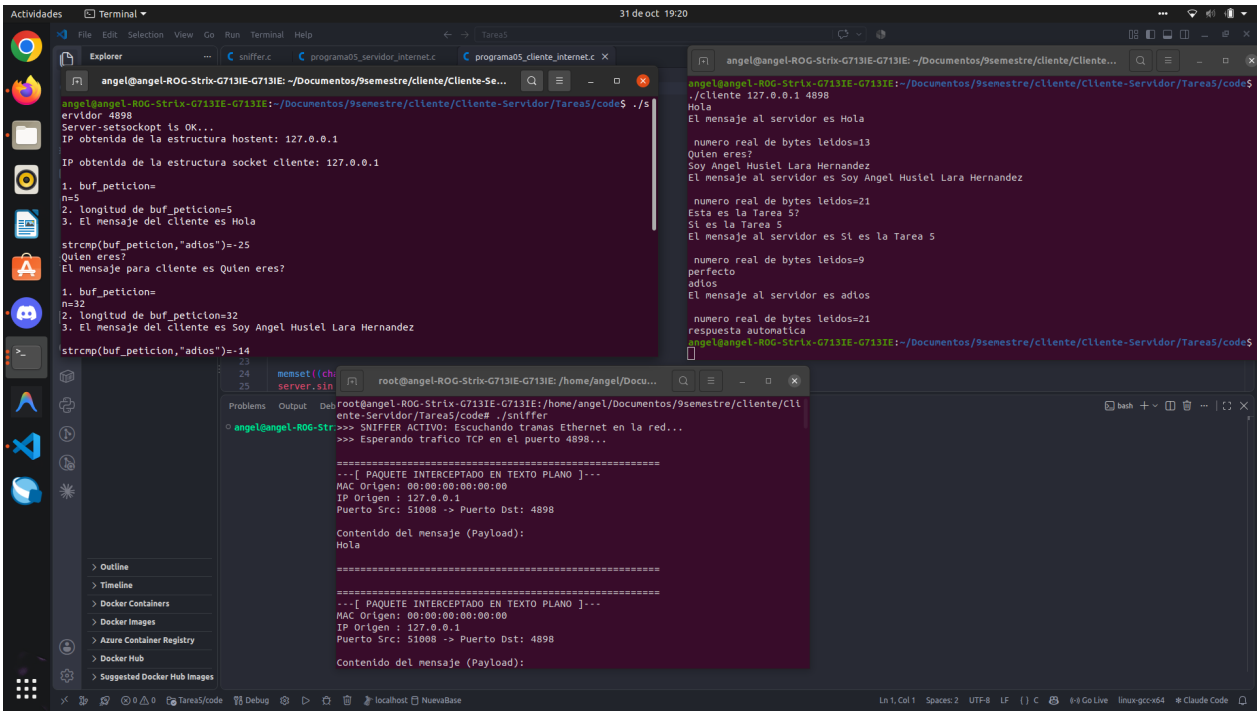


Figura 3: Intercambio de mensajes con el programa 05 y captura del mensaje Hola en el sniffer.

Para el caso de esta prueba, el cliente inició con `Hola` y el servidor respondió `Quien eres?`. Después se envió `Soy Angel Husiel Lara Hernandez`, a lo que el servidor respondió `Esta es la Tarea 5?`. El cliente contestó `Si es la Tarea 5` y recibió `perfecto`.

Lo que vemos en el sniffer es el mensaje `Hola` dentro del payload, con IP de origen `127.0.0.1`, puerto de origen 51008 y puerto de destino 4898. En este caso se capturó un mensaje enviado del cliente hacia el servidor, a diferencia de la prueba anterior.

Por último, el cliente escribió `adios` y recibió `respuesta automatica`, con lo que cerró su conexión. El servidor del programa 05 está preparado para regresar a esperar otro cliente después de terminar

ese intercambio.